

PENETRATION TEST REPORT

Application Security Assessment – Web Application

Client:	TryBankMe Limited
Assessment Type:	External Black-Box Penetration Test
Report Date:	05 August 2026
Document Version:	1.0
Classification:	CONFIDENTIAL
Prepared By:	Red Team Consultant, SecureWorks Security Co.

This document is confidential and may not be reproduced or distributed without the prior written consent of the client. All findings remain the property of TryBankMe Limited.

1. Executive Summary

1.1 Overview

1.2 Results

1.3 Impact

1.4 Remediation Direction

2. Vulnerability Write-Ups

2.1 Finding 01 – Race Condition in Transaction Handling Allows Balance Manipulation

2.2 Finding 02 – Unauthenticated SQL Injection in Login Form

3. Appendices

Appendix A – Assessment Scope

Appendix B – Assessment Artefacts

Appendix C – Styling & QA Best Practices

1. Executive Summary

The executive summary is written for business and security stakeholders. It explains what was tested, what was found, and why it matters — all in business terms.

1.1 Overview

A black-box penetration test was performed against the TryBankMe web application, an online banking platform. The assessment focused on core banking functionality including registration, authentication, and transaction processing, with the goal of identifying security weaknesses prior to public launch.

1.2 Results

The application was found to be largely well-secured, with controls for session handling and data validation exceeding baseline expectations. However, one high-severity race condition vulnerability was identified within the transaction system that, when exploited, allows an account balance to be manipulated to produce an infinite money glitch.

Severity	Count
Critical	0
High	1
Medium	1
Low	0
Informational	0

1.3 Impact

If left unaddressed, the race condition could be exploited to generate unlimited funds within user accounts. Although the flaw requires valid credentials and is constrained to transaction timing, a determined actor could repeatedly trigger the condition to inflate an account balance and subsequently withdraw or transfer the illegitimate funds, exposing the organisation to direct financial loss and regulatory scrutiny.

1.4 Remediation Direction

The identified issues are addressed with clearly scoped remediation actions that can be implemented within the existing development cycle. The race condition requires a targeted code-level fix to serialise concurrent transactions, while a secondary finding requires introduction of parameterised database queries. No major architectural investment is required. Re-testing is recommended following remediation.

2. Vulnerability Write-Ups

This section is the technical heart of the report and is written primarily for developers and system administrators responsible for remediation. Each finding follows a consistent structure: Title, Risk Rating, Summary, Background, Technical Details & Evidence, Impact, Remediation Advice and References. Each vulnerability is rated in isolation as if no other vulnerabilities existed.

2.1 Finding 01 – Race Condition in Transaction Handling Allows Balance Manipulation

Risk Rating

High (CVSS 3.1 Base Score: 8.6) – rated in isolation. Exploitation does not require an authentication bypass but does require valid user credentials.

Summary

A race condition vulnerability was identified in the transaction processing workflow of the TryBankMe application. Under concurrent requests, the application can process multiple transfers against the same opening balance, allowing an account to exceed its available funds.

Background

Race conditions occur when multiple processes or threads access and modify shared data without appropriate synchronisation. In financial systems, balance checks and deduction operations are frequently executed as separate steps. If these steps are not atomic, an attacker can submit two or more simultaneous requests, each of which reads the original balance before either deduction is committed.

Technical Details & Evidence

The transfer endpoint `/api/transfer` was observed to read the account balance, validate sufficient funds, then deduct the amount. Sending two concurrent requests while holding the initial balance state resulted in both requests succeeding:

```
POST /api/transfer HTTP/1.1
Host: trybankme.com
Content-Type: application/json

{"from": 1001, "to": 1002, "amount": 1000}
```

Both responses returned HTTP 200 with a success status. Post-transfer reconciliation showed a single deduction was lost, confirming the concurrent transactions consumed the same source balance.

Impact

An authenticated attacker could submit multiple simultaneous transfer requests to repeatedly inflate the receiving account balance without a corresponding deduction from the source account. This produces an "infinite money glitch" enabling withdrawal or transfer of funds that were never available, exposing the organisation to direct financial loss.

Remediation Advice

Transactions that read and modify account balances must be made atomic so that no two requests can operate on the same balance concurrently. This can be achieved by enforcing row-level locks during the balance read and deduction, or by serialising transfer processing through a queue. The first and primary recommendation is to ensure the balance check and deduction occur within a single atomic database transaction.

References

OWASP API Security Top 10 – Broken Object Level Authorization; CWE-362: Race Condition.

2.2 Finding 02 – Unauthenticated SQL Injection in Login Form

Risk Rating

High (CVSS 3.1 Base Score: 8.6) – rated in isolation.

Summary

An unauthenticated SQL injection vulnerability was identified in the login form of the TryBankMe application. This could allow an attacker to bypass authentication or extract sensitive customer information.

Background

SQL injection occurs when user-supplied input is unsafely included in SQL queries without proper parameterisation. The SQL interpreter is unable to discern SQL commands from the user-supplied input, allowing a threat actor to inject directly into the SQL command itself. This vulnerability is commonly exploited to bypass authentication or extract sensitive data.

Technical Details & Evidence

The login form at **/login** was vulnerable to SQL injection via the **username** field. Using Burp Suite, the following request was intercepted and modified:

```
POST /login HTTP/1.1
Host: trybankme.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 45

username=' OR 1=1--&password=randomvalue
```

The application responded with a 302 Found redirect to the authenticated dashboard, confirming that authentication was bypassed using a classic SQL injection payload.

Impact

An attacker could log in as any user without valid credentials. Additionally, error-based injection techniques could be leveraged to enumerate information stored in the underlying database, including customer records.

Remediation Advice

Parameterised queries should be used for all database operations involving user input. For example, in a .NET application using MS SQL, the login query should use parameter binding so that user input is split from the SQL command. This ensures the SQL engine cannot be confused between command and data.

References

OWASP Top 10 – A03:2021-Injection; CWE-89: SQL Injection.

3. Appendices

Appendices are especially useful for security stakeholders and future testers who may need to validate what was done, verify scope, or follow up after remediation.

Appendix A – Assessment Scope

This appendix establishes how close the assessment was to what was originally scoped in the Rules of Engagement document.

- Scope: the web application www.trybankme.com, including registration, login and transaction modules.
- Coverage: approximately 100% of the agreed in-scope functionality was assessed. No out-of-scope systems were accessed.
- Limitations: testing was performed against the staging environment. Production-specific configuration may differ slightly, and a re-assessment is recommended post-deployment.

Appendix B – Assessment Artefacts

This appendix lists changes made during testing and the recommended cleanup actions. This is vital for the blue team to distinguish pentest activity from a genuine attack.

Artefact	Location	Recommended Action
Session cookies used during testing	Browser profile	Expired on session end
Test accounts	Application database	Disable and remove
Uploaded test file (payload.txt)	/uploads/	Remove after review

Appendix C – Styling & QA Best Practices

- Write in the past tense — "The vulnerability was discovered during authentication testing."
- Do not use first-person language — avoid "I", "we", "our" and "us"; write as a neutral observer.
- Mask sensitive information — never include real passwords or private data unless authorised; blur screenshots.
- Use clean, formal phrasing — "The attacker gained unauthorised access" is preferred over "we broke in."
- Be objective and consistent — avoid exaggeration, slang, and switching between terms for the same thing.
- QA process — read your own work with fresh eyes and have a peer reviewer check that findings are understandable, actionable and professionally written.